Introducción a Ciencia de Datos

Más modelos de clasificación.
Introducción a Deep learning

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Bias vs. Variance

Bias vs. Variance

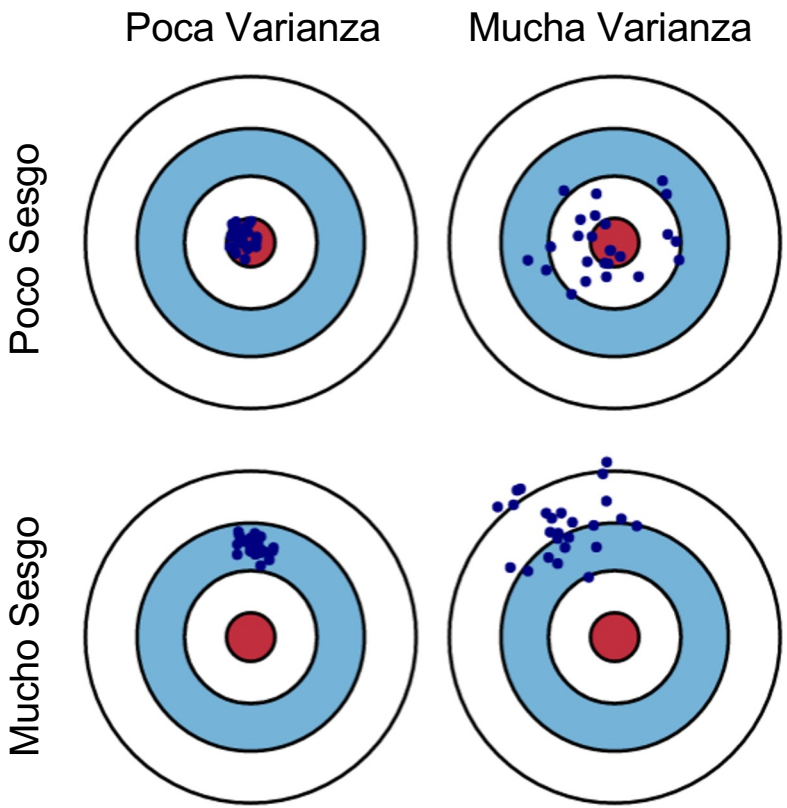
Generalization Error:
Impide que el modelo generalize bien a nuevas instancias.

$$error(X) = bias(X) + variance(X)$$

- **Bias Error**
  Suposiciones incorrectas de modelos (Sesgo Inductivo, relacionado al modelo)

- **Variance Error**
  Sensibilidad a pequeñas fluctuaciones del training set (Relacionado a los datos)

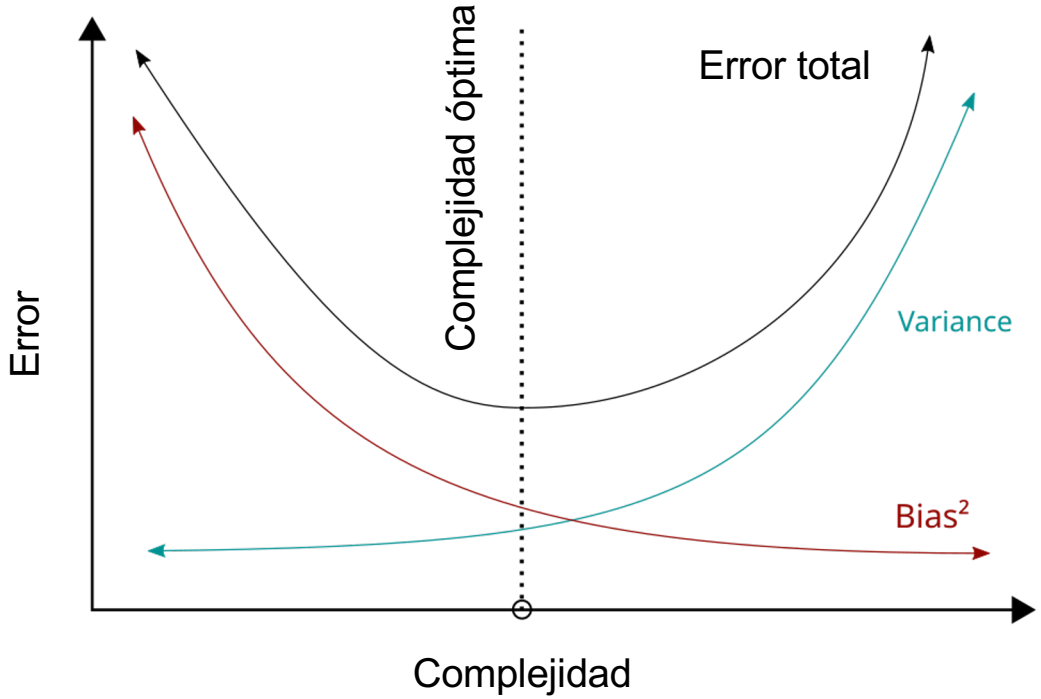
Bias vs. Variance



- Predicciones: Puntos azules.
- Target: Círculo rojo.
- Predicciones correctas caen dentro del centro.
- Bias: Distancia de las puntos azules al rojo.
- Variance: Qué tan esparcidas están las predicciones.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Bias vs. Variance Tradeoff



$$error(X) = bias(X) + variance(X)$$

Buscar punto medio

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Support Vector Machines

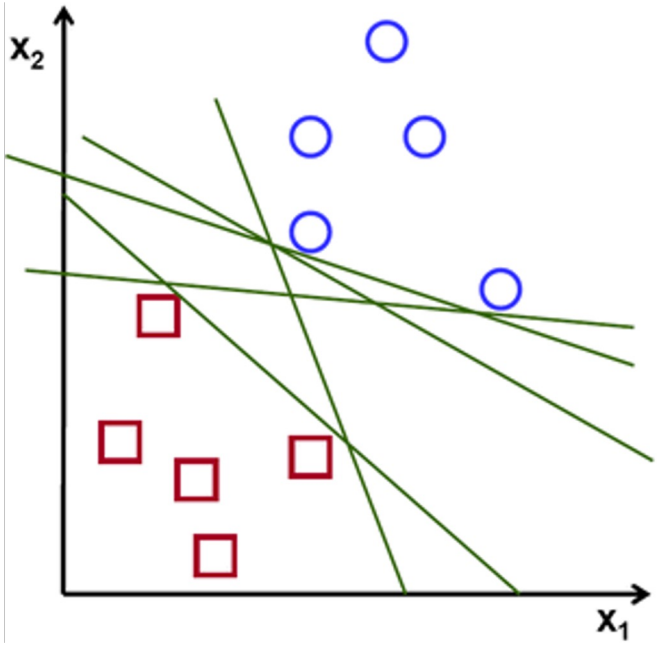
Support Vector Machines

Muy utilizados (Pre 2010) para datasets **complejos** y **chicos o medianos**.

Frontera de decisión: Superficie que **separa el espacio dimensional en dos sets**.

Múltiples decision boundaries. Cual es la correcta?

- Separación incorrecta.
- Separación con mala generalización a nuevos casos.

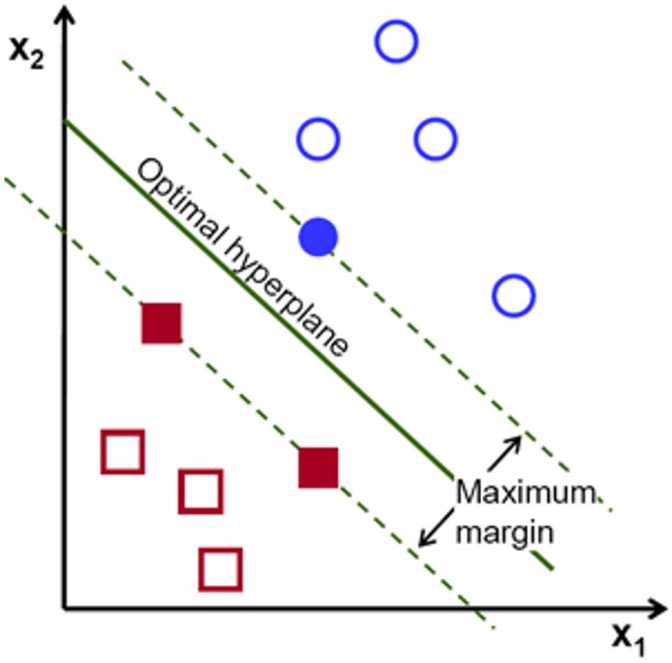


Support Vector Machines

Muy utilizados (Pre 2010) para datasets **complejos** y **chicos o medianos**.

Frontera de decisión ideal:

- Separa cada clase.
- Lo **más alejada a las instancias**.



SVM: Encontrar la ***calle* más ancha posible** entre las instancias de dos clases, a partir de **vectores de soporte** (**instancias ubicadas** en los **límites de la calle**).

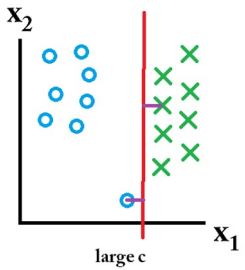
Nuevos datos fuera de la calle no modifican la frontera de decisión.

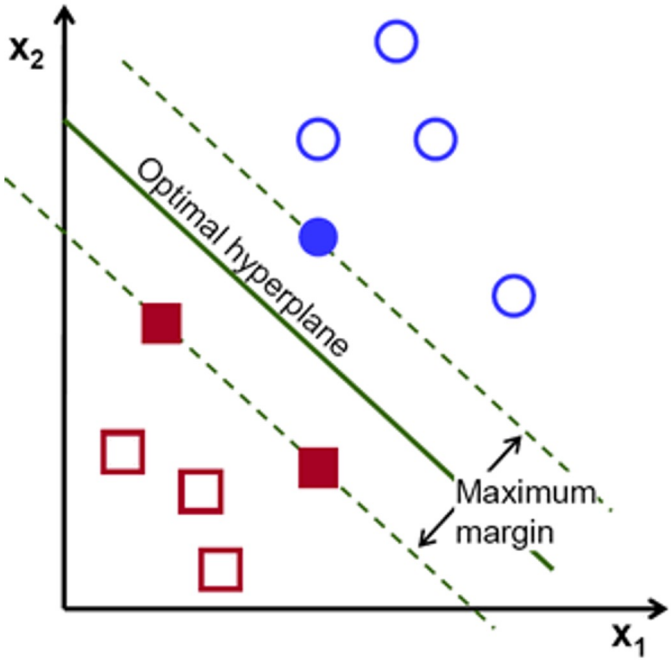
Hard Margin

- Todas las instancias se ubican del **lado correcto**.
- **No** hay instancias **dentro de la "calle"**.
- No permite *margin violations*.

Limitaciones:

- Dataset estrictamente **linealmente separable**. Bias Error!
- **Sensible a outliers**. Variance Error!





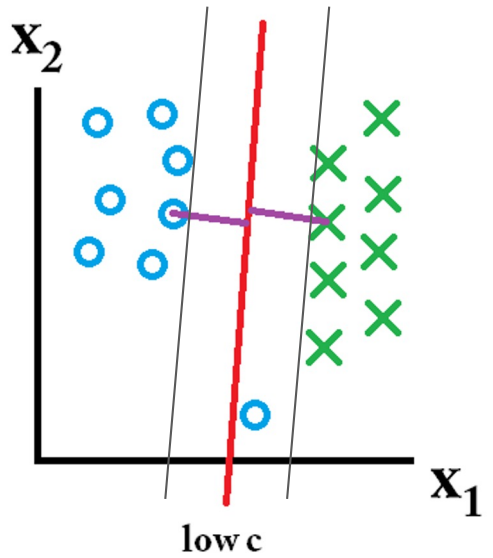
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Hard Margin

- Todas las instancias se ubican del **lado correcto**.
- **No** hay instancias **dentro de la "calle"**.
- No permite *margin violations*.

Limitaciones:

- Dataset estrictamente **linealmente separable**. Bias Error!
- **Sensible a outliers**. Variance Error!



Se resuelve si permitimos un par de errores (Soft Margin)

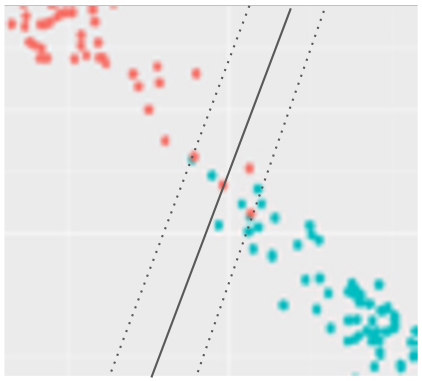
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Soft Margin

- **Balance:** Hyperparámetro de intolerancia.

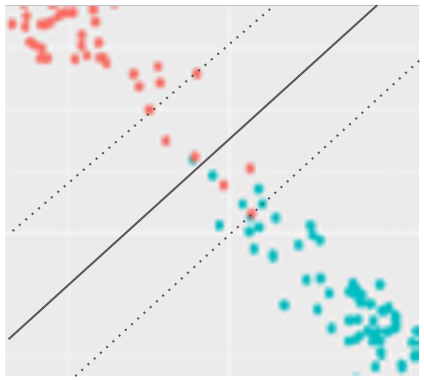
<u>Aumentar intolerancia:</u>

- Menos violaciones de márgenes.
- Calle mas delgada.



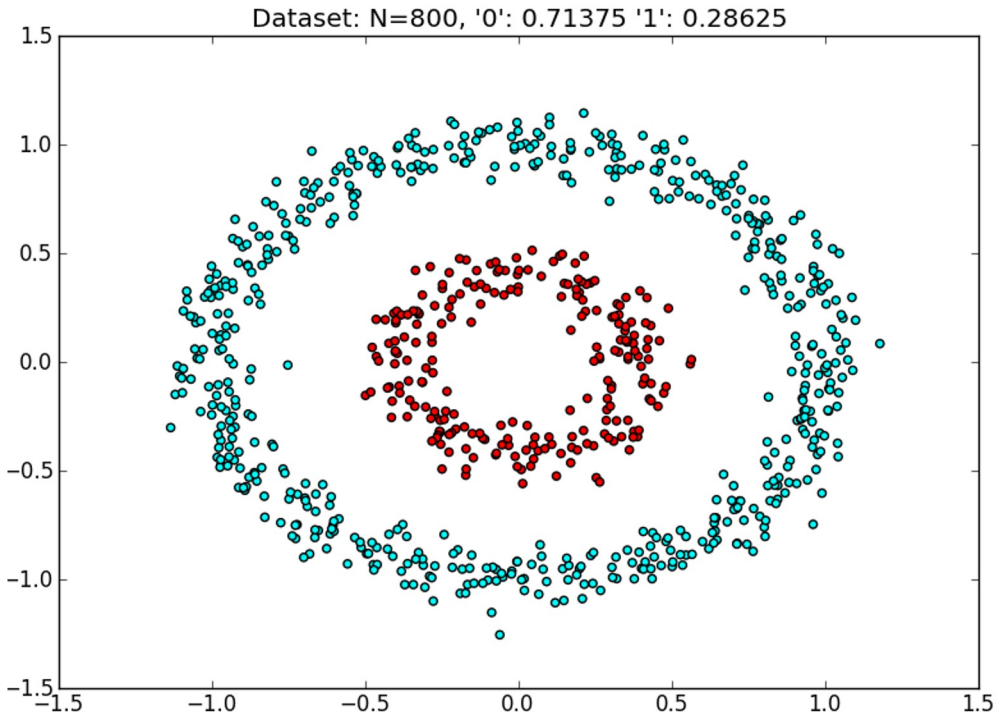
<u>Reducir intolerancia:</u>

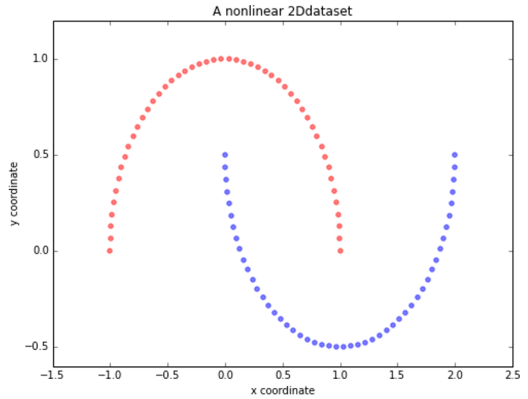
- Más violaciones de márgenes.
- Calle mas anchas.

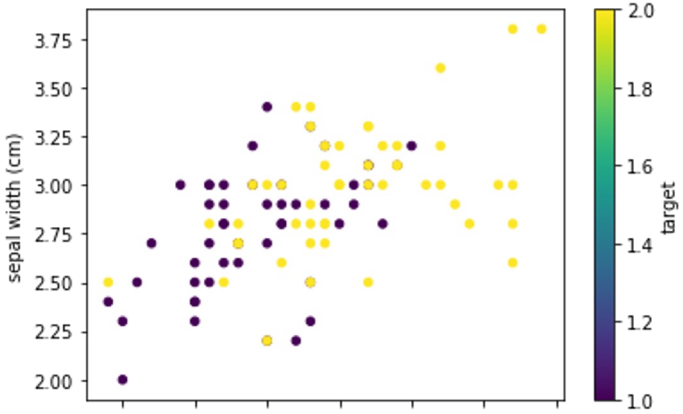


- **Overfitting:** Reducir intolerancia, mejorar generalización.

Clasificación no lineal con SVM?

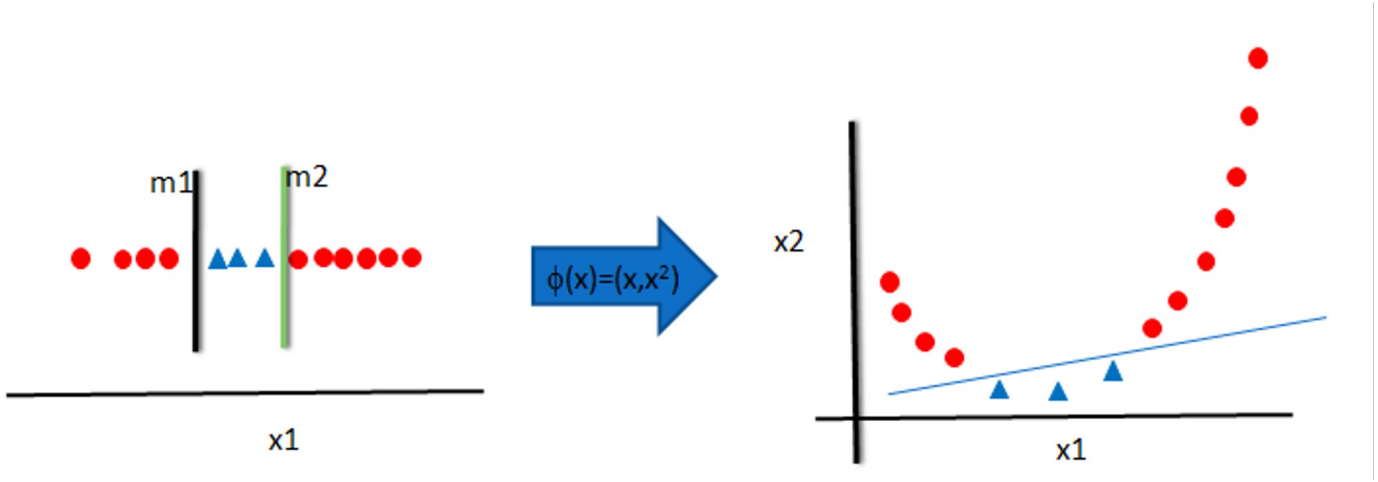




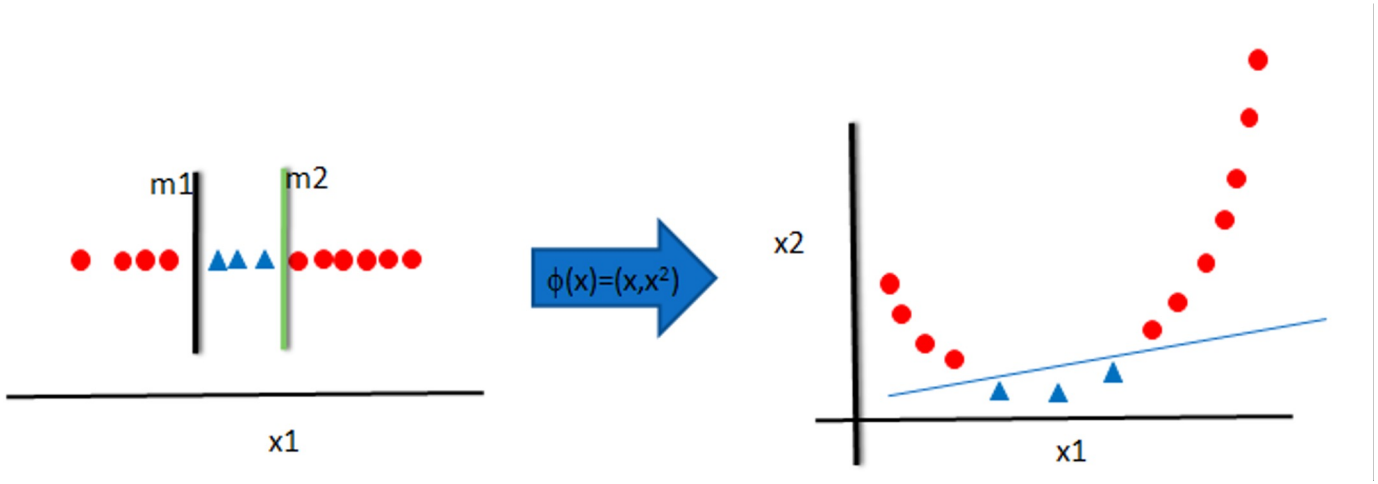


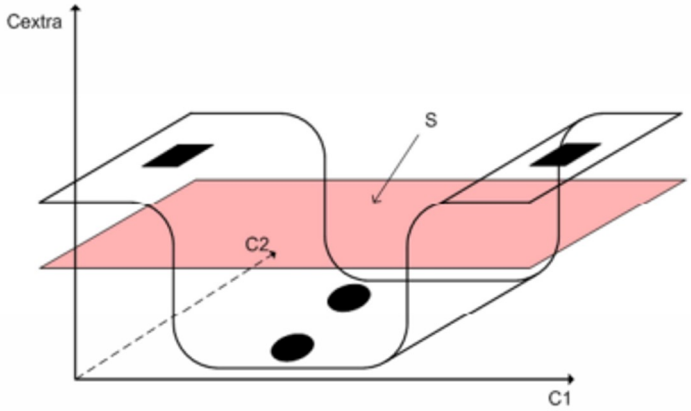
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Clasificación no lineal con SVM?



Clasificación no lineal con SVM?





Datos no separables linealmente en espacio dimensional actual, **incrementar dimensiones** en el espacio para encontrar hiperplano que los separe.

- Útil para varias situacion y algoritmos de machine learning.
- Cuidado con tener muchas features (Coste computacional)

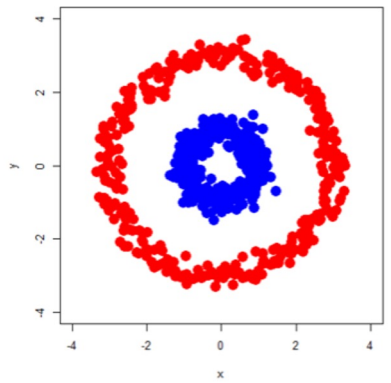
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

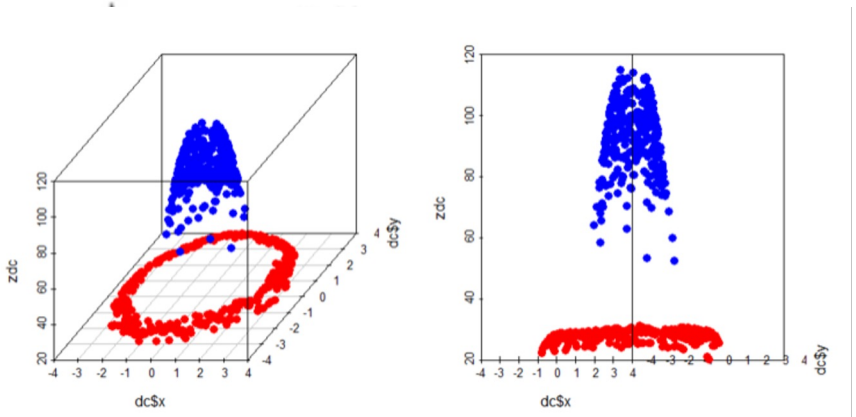
Kernels

Kernel: Función de **mapeo de un espacio dimensional a otro**.

Input space
(N dimensiones)

Feature Space
(N + M dimensiones adicionales)

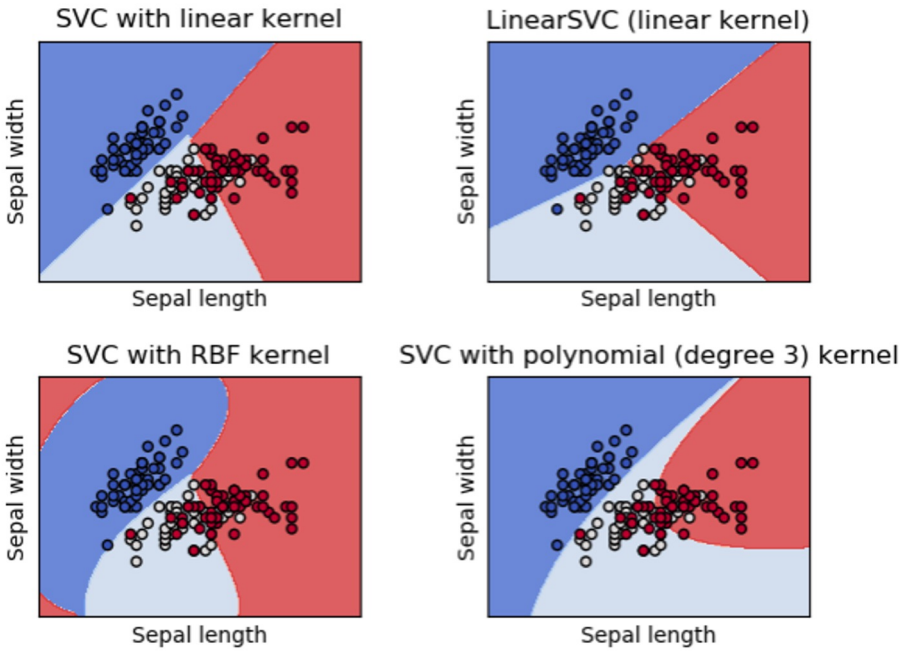




UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Kernels

Existen **diferentes kernels**: linear, polynomial, rbf y sigmoid



Elección del kernel

- Cómo regla general, probar primero el kernel lineal (el clásico)
    - Más rápido
    - Más sencillo

- Si el dataset no es demasiado extenso se puede probar Gaussian RBF

- Si hay tiempo y poder de computo:
    - Utilizar Grid Search
    - Utilizar Cross Validation

Ensambles

Ensambles

- Crear **multiples modelos**.
- **Combinarlos** para producir mejores resultados.
- Mejores métodos qué modelos individuales.

- **Averaging methods**
  Modelos independientes, promediando predicciones.
  Mejoran Variance error.
  Bagging, Random Forest

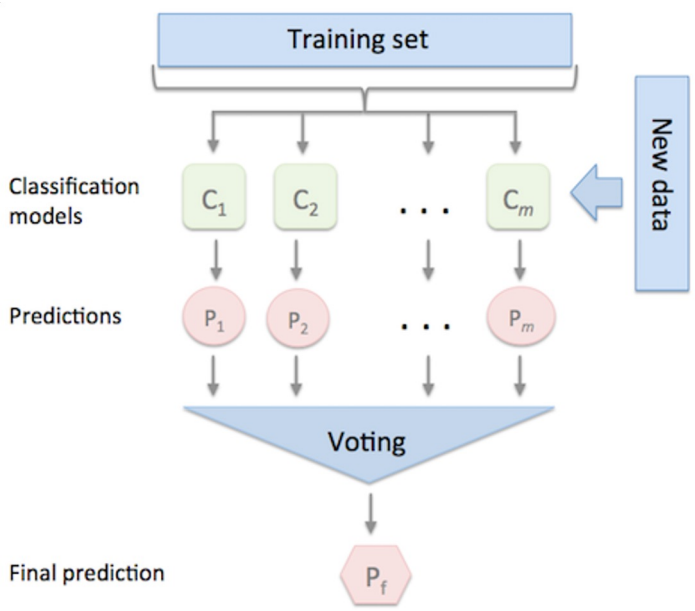
- **Boosting methods**
  Entrenamiento secuencial, potenciando debilidades del anterior.
  Mejoran Bias error.
  Adaboosting.

Voting Classifiers

Balancear debilidades individuales de modelos con performance similar.

- Crear múltiples modelos:
  - **Algoritmos distintos.**
  - **Mismo training set.**

- Predicciones:
  - Modelos reciben la misma instancia.
  - Realizan su predicciones (su voto).
  - **Votos se agregan en única predicción**.



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Formas de votar

Hard Voting: Predicción final es la más votada.

predicción = *moda*{$C_1(X)$, $C_2(X)$, $C_3(X)$ ... $C_n(X)$}

```
classifier 1 -> class A
classifier 2 -> class A
classifier 3 -> class B

predicción = moda{A, A, B} = A
```

Soft Voting: Probabilidad más peso del modelo.

$$\hat{y} = \arg\max_i \sum_{j=1}^{m} w_j p_{ij},$$

```
w = {0.2, 0.2, 0.6}
classifier              class A     class B
            class C
classifier 1            w1*0.2      w1*0.5
            w1*0.3
classifier 2            w2*0.6      w2*0.3
            w2*0.1
classifier 3            w3*0.3      w3*0.4
            w3*0.3
Average                   0.37        0.4
0.23
```

Bagging (bootstrap aggregating)

- **Reducir variance** error.
- Crear múltiples modelos **mismo algoritmo**.
- Entrenar con **subdataset random**.
- Predicciones:
  - Modelos reciben la misma instancia.
  - Realizan su predicciones (su voto).
  - Votos se agregan en única predicción:
    - **Moda (clasificacion)**
    - **Promedio (regresión)**

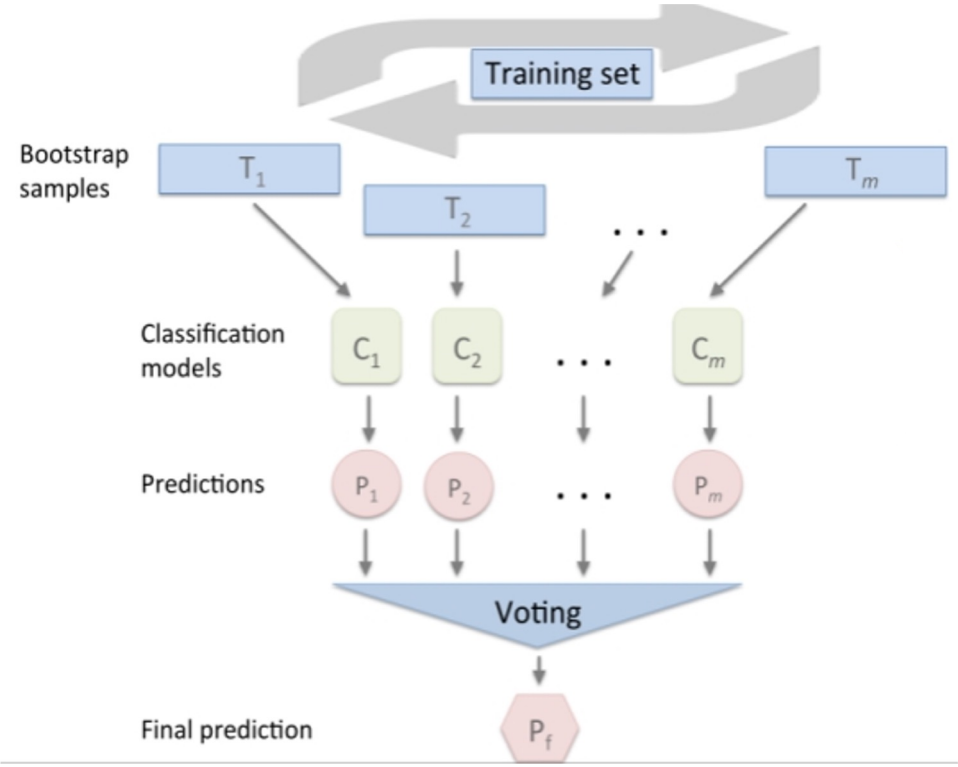
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Bagging

Bootstrap sampling:

- Generar *n* subdatasets
- Subdataset de *m* **muestras** al azar **del dataset original**:
  - **Bagging**: copiar muestra.
  - **Pasting**: cortando muestra.

n dataset de *m* muestras: Puede haber muestras duplicadas (bagging) y faltantes.



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Bagging: Ejemplo

- Sampling con reemplazo (bagging).
- $n = 3$, $m = 6$.
- Cada C_n se entrena con su subdataset.
- Clasificar nuevas instancias en cada C_n.
- Selección clase única mediante la moda.

Dataset (índices)	Bagging subset 1	Bagging subset 2	Bagging subset *n*
1	2	6	...
2	2	3	...
3	1	2	...
4	3	1	...
5	6	1	...
6	2	6	...
	C_1	C_2	C_n

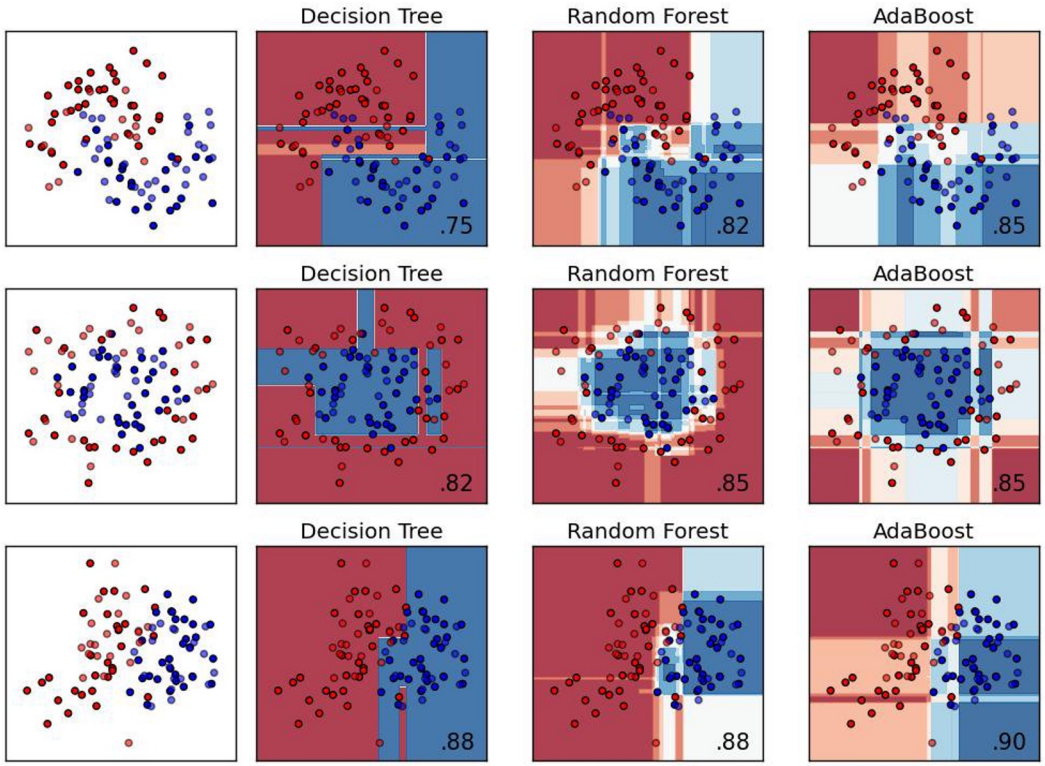
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Bagging: Ejemplo

RandomForest: *Bagging de Decision Trees*

Ensamble con bagging vs modelo individual:

- Frontera de decisión menos compleja.
- **Menor variance.**
- **Bias similar.**
- Menor overfitting. Mejor generalización a nuevos datos.



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Bagging: out-of-bag evaluation

Por cada modelo, *aún si tomamos todo el dataset con bagging*, instancias:

- Incluidas en el training set (Incluso varias veces) -> 63%.
- No utilizadas -> 37%.

Instancias **Out-of-bag**:

- varían de modelo a modelo!
- utilizar como validation set.
- promediar errores entre modelos.

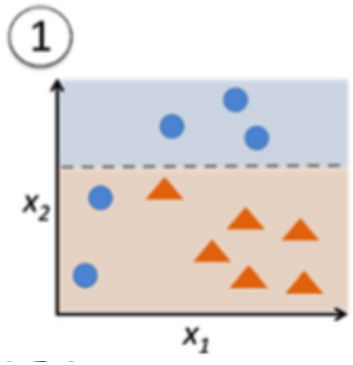
Boosting

- Convertir múltiples **modelos débiles** (similar al azar) **en fuerte**.
- **Reducir bias** error (incrementar complejidad final del ensamble).
- **Ensamblaje secuencial**:
  - Entrenar utilizando **mismo dataset y algoritmo**.
  - Modelo **sucesor corrige error del predecesor**.

Adaptive Boosting

- **Enfocarse en instancias** del dataset más **difíciles**.
- **Ajustar pesos** de la instancias **en base al error de la predicción** en la iteración **anterior**.

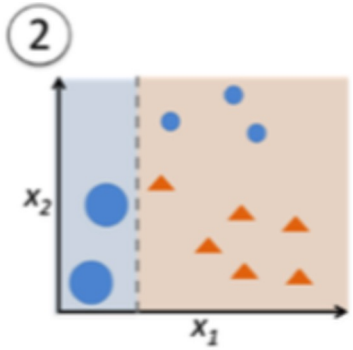
Comenzar con... un modelo débil.



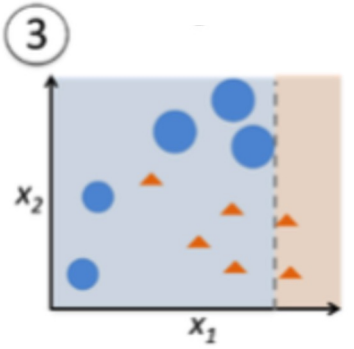
1. Instancias erróneamente clasificada, incrementar su peso.

   Ponderar modelo en base a error general sobre el dataset.

Adaptive Boosting



2. Entrenar un segundo modelo con instancias erróneas ponderadas (mayor peso).
   Nuevas instancias erróneas, volver a ajustar pesos.
   Ponderar modelo en base a error general sobre el dataset.

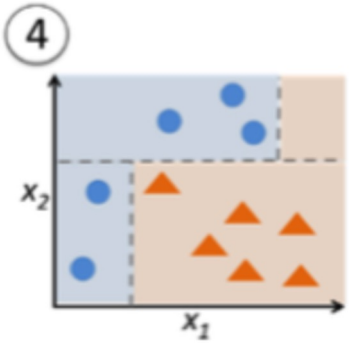


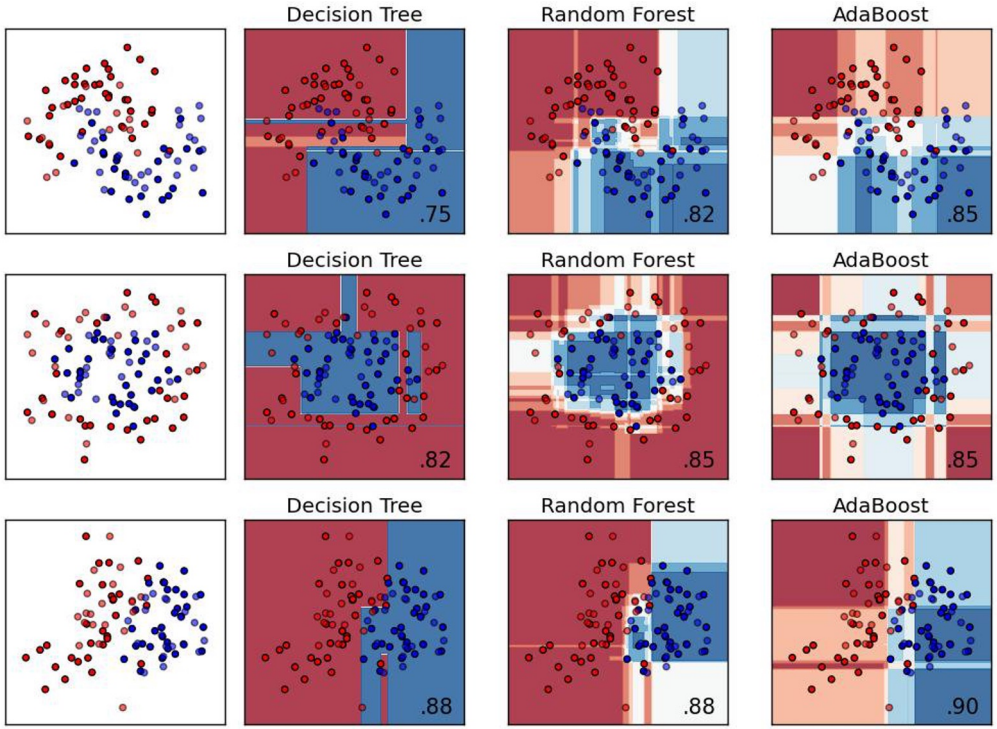
2. Entrenar nuevo modelo con instancias ponderadas.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Adaptive Boosting

Eventualmente... alcanzan *n* modelos generados o modelo perfecto.



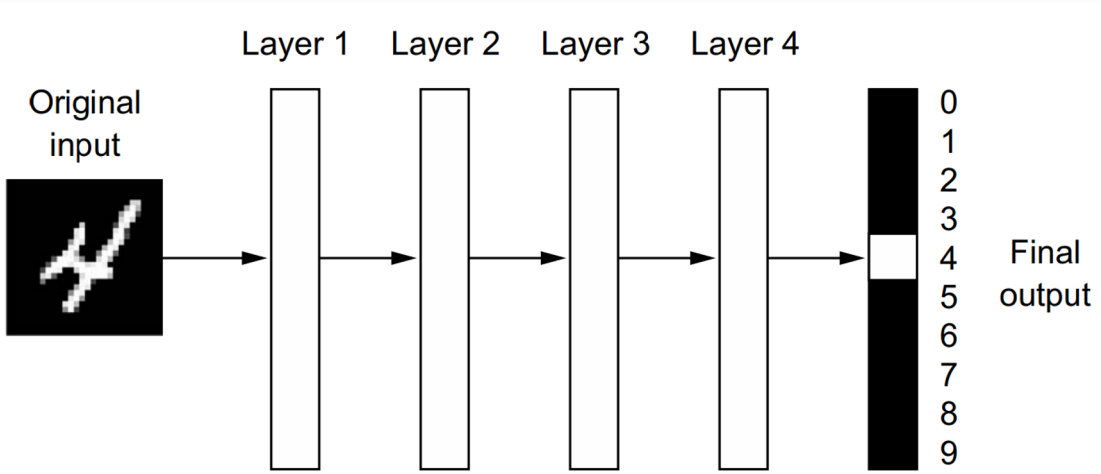


- Predicciones: Votación ponderada por peso del modelo.
- Bias Error se **reduce**.
- **Modelo fuerte** con **mejor** capacidad de **predicción qué** componentes **individuales**.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Deep learning

Qué es el Deep learning



- Es una rama del machine learning que **utiliza redes neuronales con múltiples capas** (profundas).
- En el aprendizaje profundo se busca que el modelo aprenda a *representar* las features.
- Procesa **grandes volúmenes de datos** y puede **automatizar tareas complejas** como reconocimiento de voz o imágenes.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Qué es el Deep learning

- Aunque el aprendizaje profundo es un campo antiguo (1980, **Nobel de física actual**), ganó prominencia en la década de 2010, logrando una revolución en resultados perceptuales y procesamiento de lenguaje natural.

- Ha permitido avances en áreas históricamente difíciles, incluyendo:
    - Clasificación de imágenes a nivel casi humano.
    - Transcripción de voz a nivel casi humano.
    - Transcripción de escritura a mano a nivel casi humano.
    - Mejora drástica en traducción automática.
    - Mejora drástica en conversión de texto a voz.
    - Conducción autónoma a nivel casi humano.
    - Mejoras en segmentación de anuncios.
    - Capacidad para responder preguntas en lenguaje natural.

Deep learning vs Machine Learning

Machine Learning:

- Se basa en modelos más simples que **requieren extracción manual de características**.
- Los modelos más comunes incluyen regresión lineal, árboles de decisión, y SVM.
- **Ejemplo:** En reconocimiento de imágenes, el ingeniero de datos debe seleccionar características como bordes, formas, o colores, y el algoritmo aprende a usarlas para la predicción.
- **Limitaciones:** Menor capacidad de aprender patrones complejos y no escala bien con datos no estructurados o grandes volúmenes de información.

Deep learning vs Machine Learning

Deep Learning:

- Utiliza redes neuronales profundas que **pueden aprender representaciones complejas de los datos por sí mismas**, sin necesidad de ingeniería de características.
- Puede trabajar con **grandes volúmenes de datos y datos no estructurados** (como imágenes, videos o texto).
- **Ejemplo:** En reconocimiento de imágenes, una red neuronal profunda puede aprender automáticamente características como bordes, formas y patrones sin intervención humana.

Deep learning: Ventajas y desventajas

Ventajas:

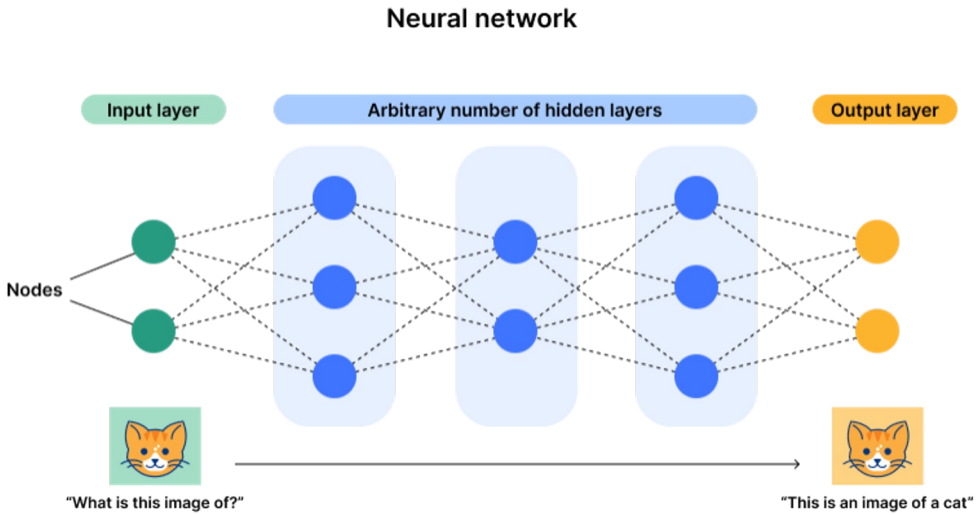
- **Escalabilidad:** Mejora el rendimiento a medida que crecen los datos.
- **Versatilidad:** Funciona bien con datos no estructurados (imágenes, texto, video).
- **Automatización de características:** No requiere ingeniería manual de características.

Desventajas:

- Necesita grandes cantidades de datos y poder computacional (GPUs).
- Más difícil de interpretar que los modelos de machine learning tradicionales.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

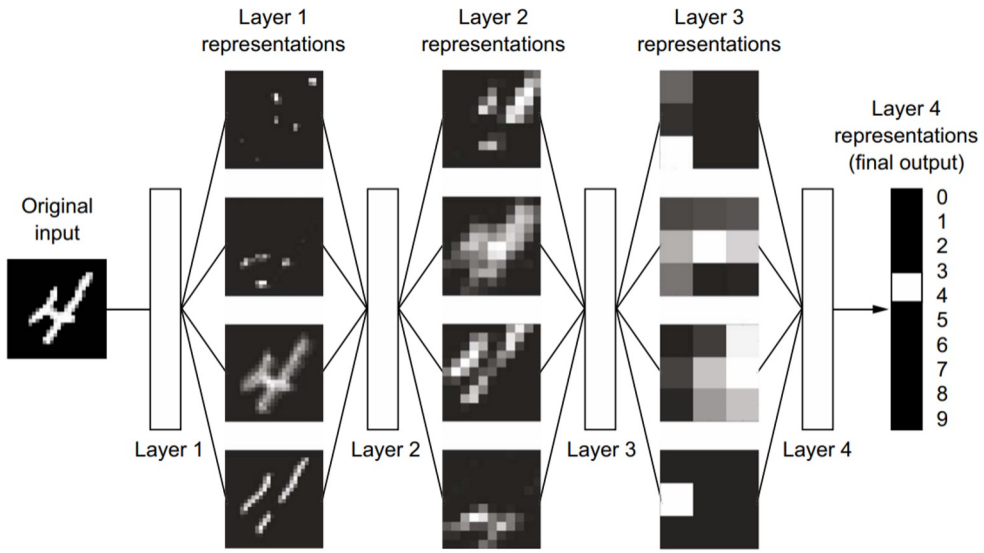
Cómo Funciona una Red Neuronal



- Una red neuronal está **formada por capas de nodos** llamados **neuronas**:
  - **Capa de entrada:** Recibe los datos de entrada.
  - **Capas ocultas:** Procesan los datos en múltiples etapas (más capas ocultas = red más profunda). Puede tener tantas neuronas como uno quiere.
  - **Capa de salida:** Entrega el resultado final (por ejemplo, la clasificación de una imagen).

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Cómo Funciona una Red Neuronal



- La red transforma la **imagen digital en representaciones** que son cada vez más diferentes de la imagen original y cada vez más informativas sobre el resultado final.

- Se puede pensar en una red profunda como un **proceso de destilación de información en múltiples etapas**, donde la información pasa a través de filtros sucesivos y sale cada vez más purificada.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Componentes de una red

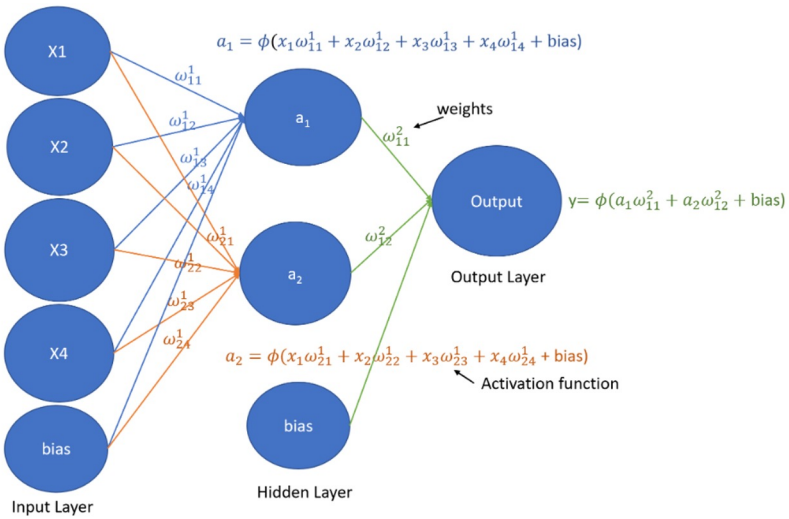
Ingredientes para preparar una red neuronal

- 1 capa de entrada.
- 1 capa de salida.
- N (cuantas guste) capas ocultas (*fully conected* ***por ahora***)
- Pesos y bias.
- Funciones de activación.
- Función de pérdida

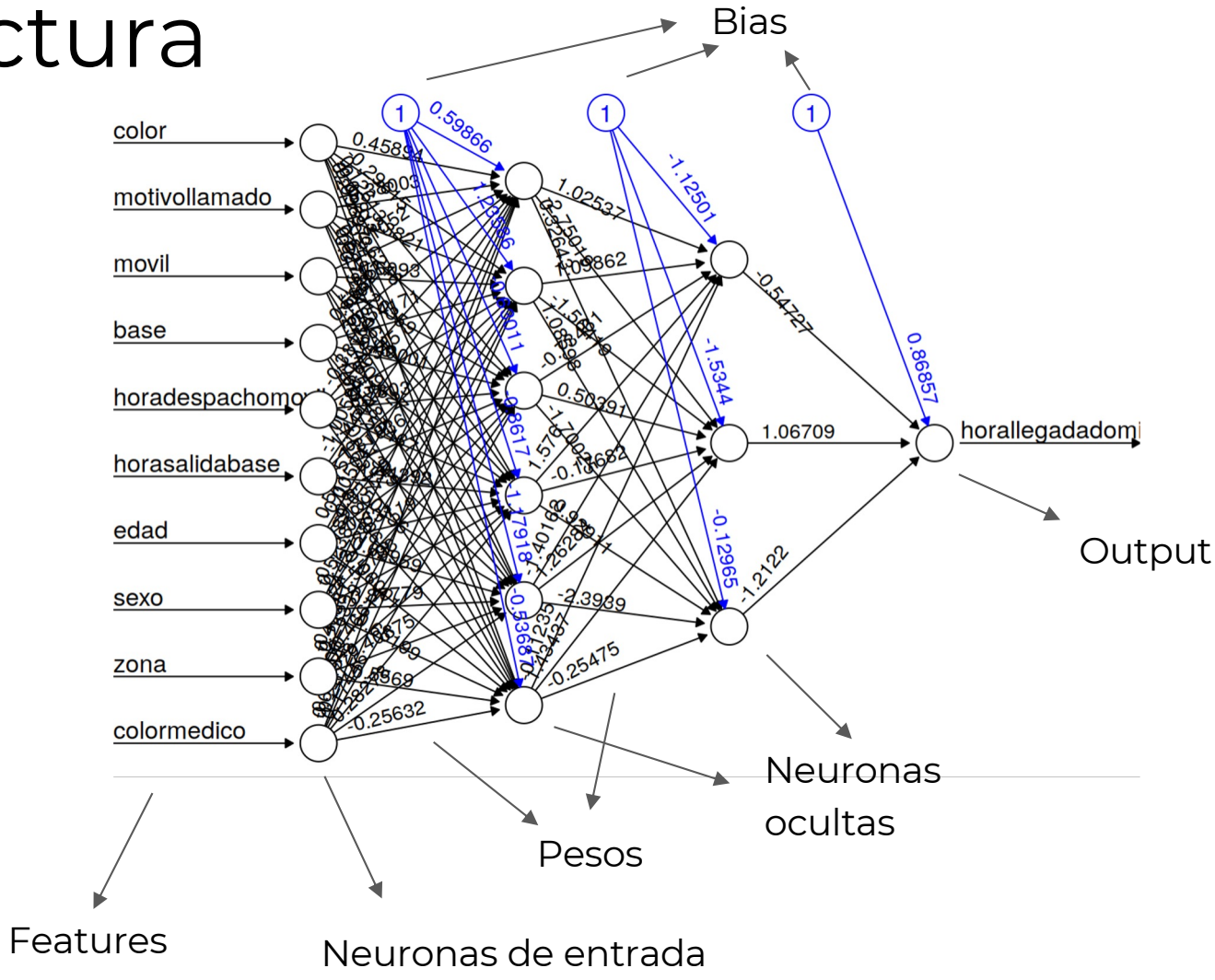
Pesos y bias

Componentes de cada neurona:

- **Pesos (W):**
  - Cada conexión entre las neuronas tiene un peso asociado que indica cuánta **influencia tiene una neurona en otra**.
  - Se actualizan durante el proceso de entrenamiento usando técnicas para minimizar el error.
- **Bias (b):**
  - Cada neurona tiene un bias que le permite ajustar el valor de salida, independientemente de las entradas.
  - Ayuda a que el modelo sea más flexible y ajuste mejor los datos.



Estructura

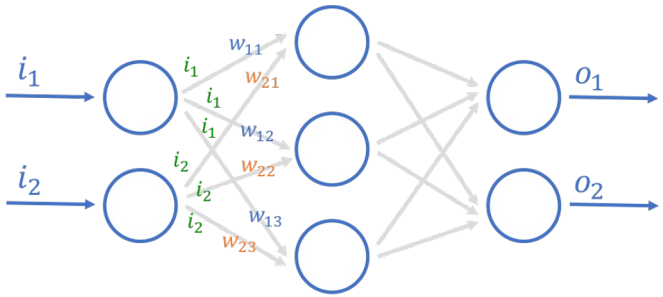


UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Matrices!

- Todos estos pesos pueden ser representados como **matrices**, y cada **estado** (los valores que toman las neuronas de una capa al pasarle una instancia) pueden ser representados como un **vector**.

- Esto es **CLAVE** para el entrenamiento de redes de manera rápida (GPUs).



Cálculo del *estado oculto*

$$\begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \end{bmatrix} \cdot \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} = \begin{bmatrix} (w_{11} \times i_1) + (w_{21} \times i_2) \\ (w_{12} \times i_1) + (w_{22} \times i_2) \\ (w_{13} \times i_1) + (w_{23} \times i_2) \end{bmatrix}$$

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

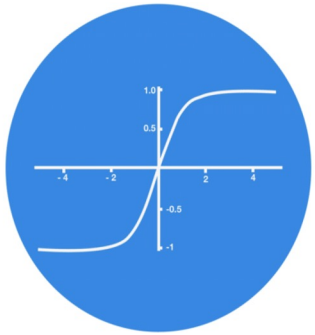
Funciones de activación

- Las funciones de activación **transforman la salida de una neurona** antes de enviarla a la siguiente capa.

- Tipos comunes de funciones de activación:

    - **ReLU (Rectified Linear Unit):** Convierte valores negativos en 0 y deja los positivos igual. Usada en la mayoría de las redes profundas porque facilita el entrenamiento.
    - **Sigmoid:** Convierte la salida en un valor entre 0 y 1. Común en la última capa de modelos de clasificación binaria.
    - **Softmax:** Generaliza Sigmoid para problemas multiclase. Produce una distribución de probabilidad sobre las clases posibles.

5
0.1
-0.5

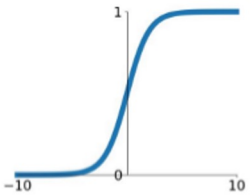


UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Funciones de activación

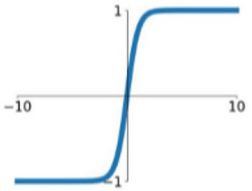
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



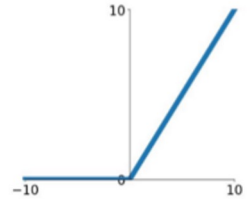
tanh

$$\tanh(x)$$



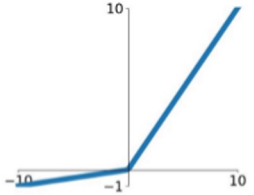
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

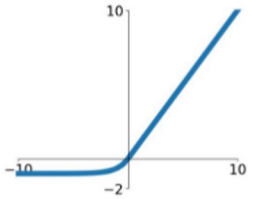


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

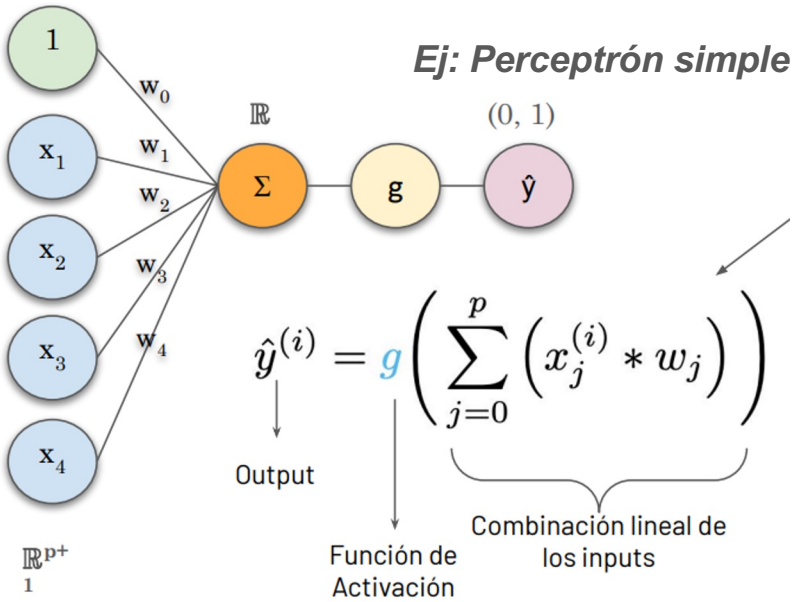
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Funciones de activación

- Una función de activación es crucial en una red neuronal porque introduce no linealidad en el modelo, lo que le permite aprender y representar relaciones complejas entre las variables de entrada y la salida.
- Sin una función de activación, la red neuronal sería simplemente una combinación lineal de las entradas, lo que la limitaría a resolver solo problemas lineales, independientemente del número de capas o neuronas.
- Además nos permite modelar problemas de clasificación (lo vemos en un rato)

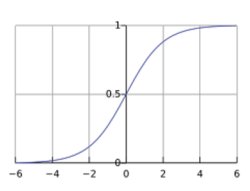


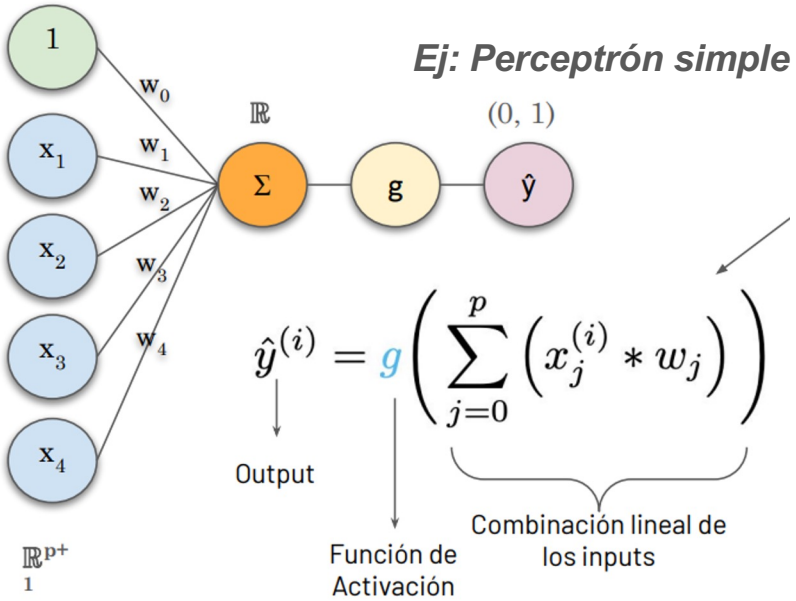
Paréntesis: Clasificación

Que pasa si queremos un clasificador binario?

- Necesitamos una **PROBABILIDAD** (0, 1)!
- Para eso usamos una función de activación en la **capa de salida**.
- Esa función de activación debe transformar un número real (-inf, inf) a un número entre 0 y 1. El estándar es usar la sigmoidea:

$$g(z) = \frac{1}{1 + e^{-z}}$$

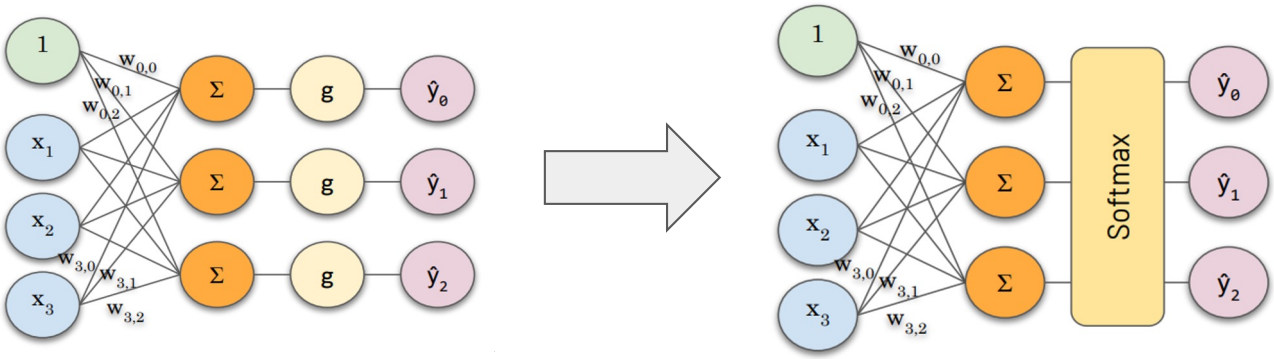


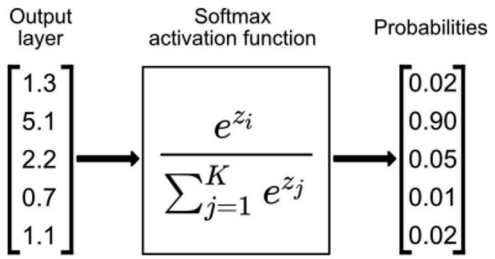


Paréntesis: Más de un output

Que pasa si queremos un clasificador multiclase?

- Necesitamos que la suma de todas las probabilidades de 1.
- Usamos una función de activación **Softmax**





Funciones de Pérdida:

Miden la **discrepancia** entre la predicción de la red y el valor real.

Son clave para poder pedir que tan bien anda nuestro modelo **durante el entrenamiento.**

Elección de la función de pérdida:

- **<u>Regresión:</u>**
    - **MSE:** Si los errores grandes deben ser fuertemente penalizados.
    - **MAE:** Si queremos penalizar uniformemente todos los errores, sin que los valores atípicos influyan demasiado.
    - **Huber:** Cuando necesitamos un equilibrio entre penalización fuerte y suavidad en errores grandes.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Funciones de Pérdida:

- **Clasificación Binaria:**
    - **Binary Cross-Entropy:** La opción más común para problemas de clasificación binaria, como la detección de fraude o diagnóstico médico.
- **Clasificación Multiclase:**
    - **Categorical Cross-Entropy:** Ideal cuando hay múltiples clases exclusivas (una muestra pertenece solo a una clase).
    - **Sparse Categorical Cross-Entropy:** Se usa en escenarios con muchas clases, como el reconocimiento de imágenes con una gran cantidad de categorías

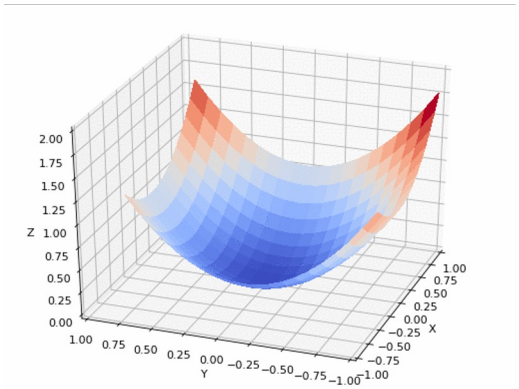
Descenso de gradiente y backpropagation

Cómo aprende una red neuronal?

Hasta acá vimos lo necesario para definir una red neuronal (Sus capas, sus funciones de activación y la función de pérdida).

Pero, cómo hacemos que esto más los datos nos de algo que pueda predecir cosas?

Algoritmo de Backpropagation y descenso de gradiente!



Descenso de gradiente

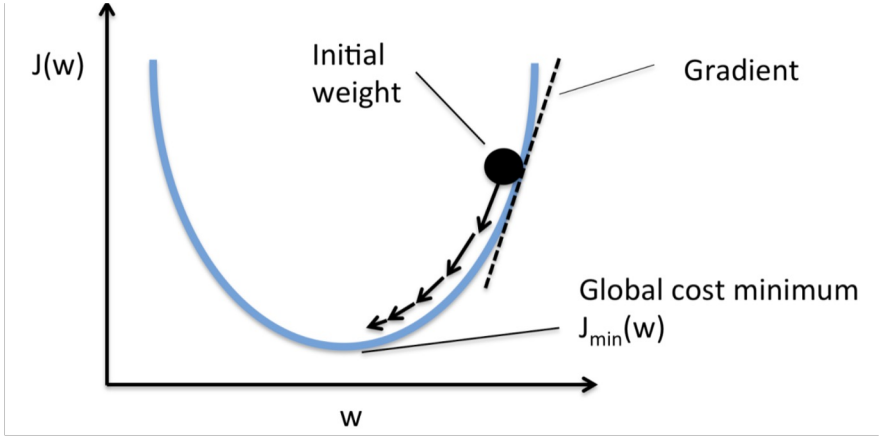
El método más utilizado para optimizar los pesos en la red.

1. **Extracción de Datos:**
   - Tomamos un lote de muestras de entrenamiento *X* y sus objetivos correspondientes *y_true*.
2. **Predicción:**
   - Realizamos una pasada hacia adelante *(forward pass)* para obtener predicciones *y_pred*
3. **Cálculo de Pérdida:**
   - Medimos la pérdida *(loss)*, que indica qué tan lejos están *y_pred* de *y_true*
4. **Ajuste de Pesos:**
   - Actualizamos los pesos para reducir la pérdida. Este es el paso clave en el entrenamiento.

Probar diferentes valores manualmente para cada peso sería ineficiente...

Descenso de gradiente

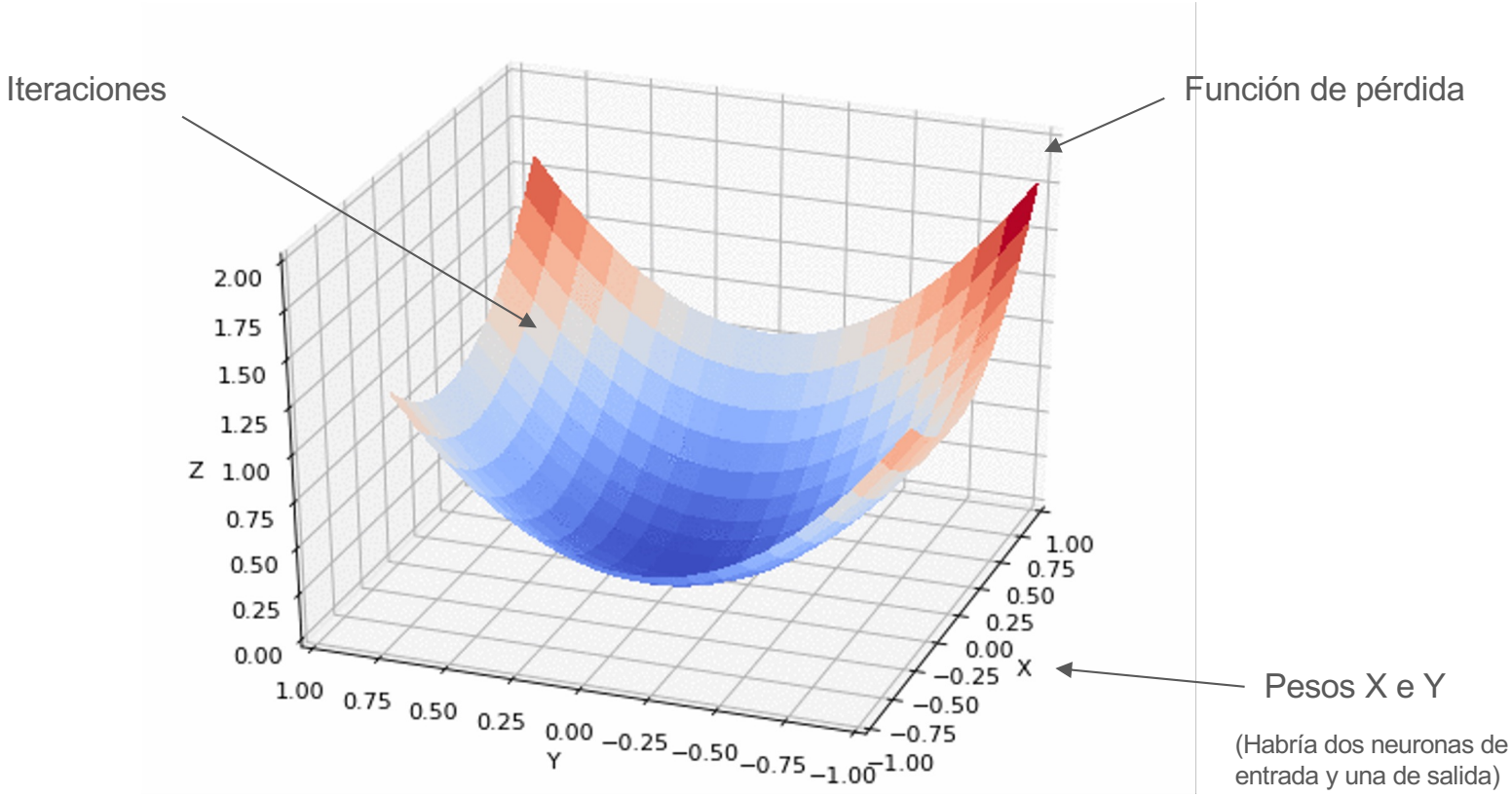
- **Actualización de Pesos:**
  - Usamos el *descenso de gradientes* para mover todos los pesos en la dirección que minimiza la pérdida, permitiendo una actualización simultánea y eficiente.



El entendimiento del algoritmo de Backprogation requiere de derivadas parciales y conocer AST. Les dejamos documentación para que puedan leer los más interesados.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

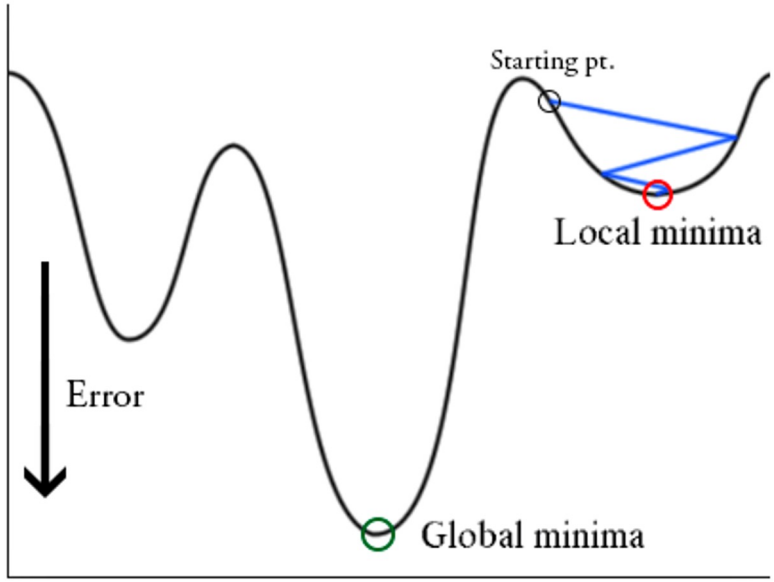
Descenso de gradiente



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Descenso de gradiente

- Puede ser **computacionalmente costoso** cuando se trabaja con grandes conjuntos de datos, ya que cada iteración requiere evaluar el modelo con todos los datos.
- Las funciones de pérdida pueden tener **múltiples mínimos locales** y el descenso de gradientes **puede quedar atrapado** en uno de estos, sin encontrar el mínimo global que corresponde al mejor ajuste.



Learning Rate (el ingrediente secreto)

El Learning Rate (tasa de aprendizaje) es un **hiperparámetro** clave en el entrenamiento de redes neuronales.

Controla el tamaño de los pasos que el **optimizador** da en la dirección del gradiente al ajustar los pesos del modelo.

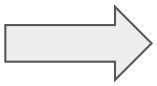
- **Tasa alta:** Da pasos grandes, lo que puede hacer que el modelo converja más rápido, pero con el riesgo de saltarse el mínimo óptimo o volverse inestable.
- **Tasa baja:** Da pasos pequeños, lo que permite una convergencia más precisa pero puede hacer que el entrenamiento sea muy lento o que quede atrapado en mínimos locales.

Hay maneras de controlarlo **DURANTE** el entrenamiento.

Descenso de gradientes estocástico

- En lugar de utilizar todos los datos de entrenamiento, **elige un único dato o un pequeño lote** (mini-batch) de ejemplos de entrenamiento de manera aleatoria para calcular el gradiente y actualizar los parámetros.
- Es **más rápido** en cada iteración, pero el gradiente que calcula es más ruidoso (menos preciso) debido a la aleatoriedad de las muestras.
- **Puede escapar de mínimos locales** y zonas planas con mayor facilidad debido a la naturaleza estocástica del algoritmo.
- Se utiliza ampliamente en la práctica debido a su eficiencia computacional y porque logra converger de manera más rápida en grandes conjuntos de datos.

Función de pérdida calculada sobre dataset de N instancias
(1 época es un ajuste sobre todo el dataset)

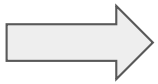


Función de pérdida calculada sobre una muestra del dataset de M << N instancias
(varias actualizaciones de pesos por época)

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Descenso de gradientes estocástico

Función de pérdida calculada sobre dataset de N instancias
(1 época es un ajuste sobre todo el dataset)



Función de pérdida calculada sobre una muestra del dataset de M << N instancias
(varias actualizaciones de pesos por época)

Cada una de estas muestras es un batch (lo van a encontrar en la literatura como **Minibatch Gradient Descent**)

La idea es que, como cambian los datos sobre los que cálcula la función de perdida, cambia la forma de la misma (el mínimo está en otro lado) haciendo que la red no converja a mínimos locales tan facilmente.

Tensorflow

Introducción a TensorFlow



- **TensorFlow** es una biblioteca de código abierto desarrollada por Google para realizar tareas de machine learning y deep learning.
- Ofrece una amplia gama de herramientas para construir y entrenar **modelos de redes neuronales.**
- Es altamente **flexible y escalable**, lo que permite su uso en proyectos pequeños y en grandes implementaciones de producción.
- Soporte para **CPU y GPU** (no van a querer entrenar redes en CPU igualmente)

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Otros Frameworks

- **Pytorch** y **JAX** son otros Frameworks muy utilizados en el desarrollo de modelos de Deep Learning, pero han proliferado más en el ambiente académico dado que permiten mayor control de cómo se funciona la red por dentro.

- En general son menos limitados que **Tensorflow** pero requieren un entendimiento más detallado de redes neuronales.

- **JAX** está especialmente pensado para utilizarse en Hardware específico, donde es mucho más rápido que los otros frameworks.

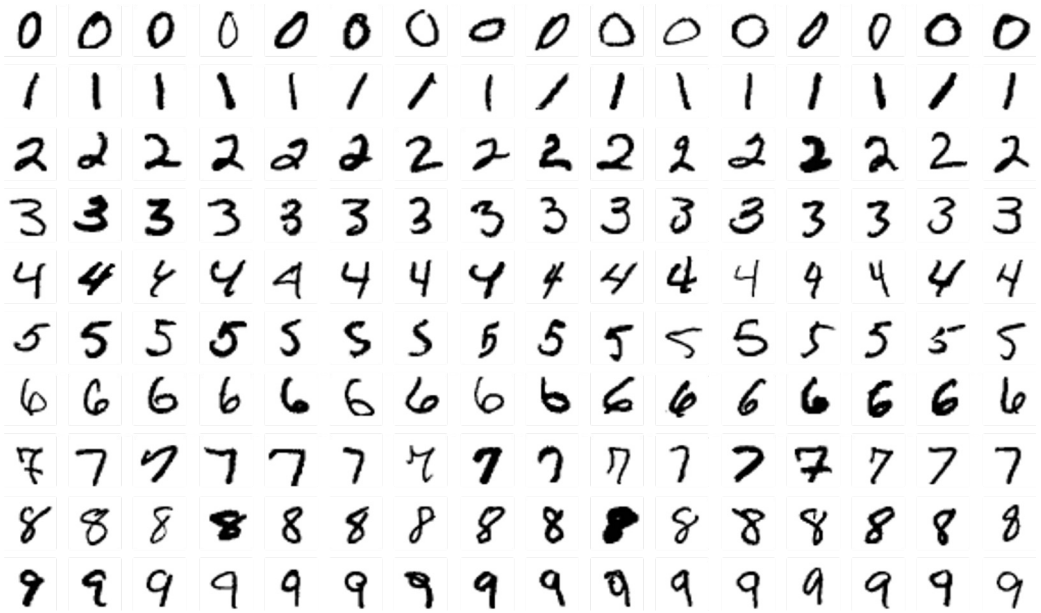




MNIST - handwritten digits

```python
# Importar el dataset MNIST
from tensorflow.keras.datasets import mnist

# Cargar el dataset y dividirlo en conjunto de entrenamiento y prueba
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```



- Las imágenes están codificadas como arrays de NumPy, y las etiquetas son un arreglo de dígitos que van del 0 al 9.
- Las imágenes y las etiquetas tienen una correspondencia uno a uno.

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

La arquitectura de la red

```python
# Importar las capas necesarias de Keras
from tensorflow.keras import layers, models

# Definir el modelo secuencial
model = models.Sequential([
    layers.Flatten(input_shape=(28, 28)),  # Aplanar las imágenes de 28x28 a un vector
    layers.Dense(512, activation='relu'),  # Capa oculta con 128 neuronas y función de activación ReLU
    layers.Dense(10, activation='softmax')  # Capa de salida con 10 neuronas (una por clase) y Softmax
])
```

- Sequential permite definir fácilmente una arquitectura entregando una lista con las capas a utilizar.

- Las **capas densas** son las “completamente conectadas” que venimos viendo.

Compilador del modelo

- Para preparar el modelo para el entrenamiento, necesitamos elegir tres cosas como parte del paso de **compilación**:
    - **Optimizador:** el mecanismo a través del cual el modelo se actualizará según los datos de entrenamiento que vea, para mejorar su rendimiento.
    - **Función de pérdida:** cómo podrá el modelo medir su rendimiento en los datos de entrenamiento y, por lo tanto, cómo podrá orientarse en la dirección correcta.
    - **Métricas** (durante el entrenamiento y la prueba): aquí, solo nos importará la precisión (la fracción de imágenes que fueron clasificadas correctamente).

```
# Compilar el modelo
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Ajuste del modelo (Fitting)

- Ahora estamos listos para entrenar el modelo, lo cual en Keras se realiza a través de una llamada al método fit() del modelo—ajustamos el modelo a sus datos de entrenamiento.
- Durante el entrenamiento, se muestran dos cantidades: la pérdida del modelo sobre los datos de entrenamiento y la precisión del modelo sobre los datos de entrenamiento.

```
[ ] # Entrenar el modelo
    history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 5ms/step - accuracy: 0.9027 - loss: 0.3356 - val_accuracy: 0.9715 - val_loss: 0.0957
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.9758 - loss: 0.0798 - val_accuracy: 0.9742 - val_loss: 0.0793
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.9833 - loss: 0.0532 - val_accuracy: 0.9772 - val_loss: 0.0795
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 20s 6ms/step - accuracy: 0.9889 - loss: 0.0338 - val_accuracy: 0.9817 - val_loss: 0.0642
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 20s 6ms/step - accuracy: 0.9919 - loss: 0.0253 - val_accuracy: 0.9811 - val_loss: 0.0629
```

Ajuste del modelo (Fitting)

```
>>> test_digits = test_images[0:10]
>>> predictions = model.predict(test_digits)
>>> predictions[0]
array([1.0726176e-10, 1.6918376e-10, 6.1314843e-08, 8.4106023e-06,
       2.9967067e-11, 3.0331331e-09, 8.3651971e-14, 9.9999106e-01,
       2.6657624e-08, 3.8127661e-07], dtype=float32)
```

- Cada número del índice i en ese arreglo corresponde a la probabilidad de que la imagen del dígito test_digits[0] pertenezca a la clase i. Este primer dígito de prueba tiene la puntuación de probabilidad más alta (0.99999106, casi 1) en el índice 7, así que, según nuestro modelo, debe ser un 7:

```
>>> predictions[0].argmax()
7
>>> predictions[0][7]
0.99999106
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Ajuste del modelo (Fitting)

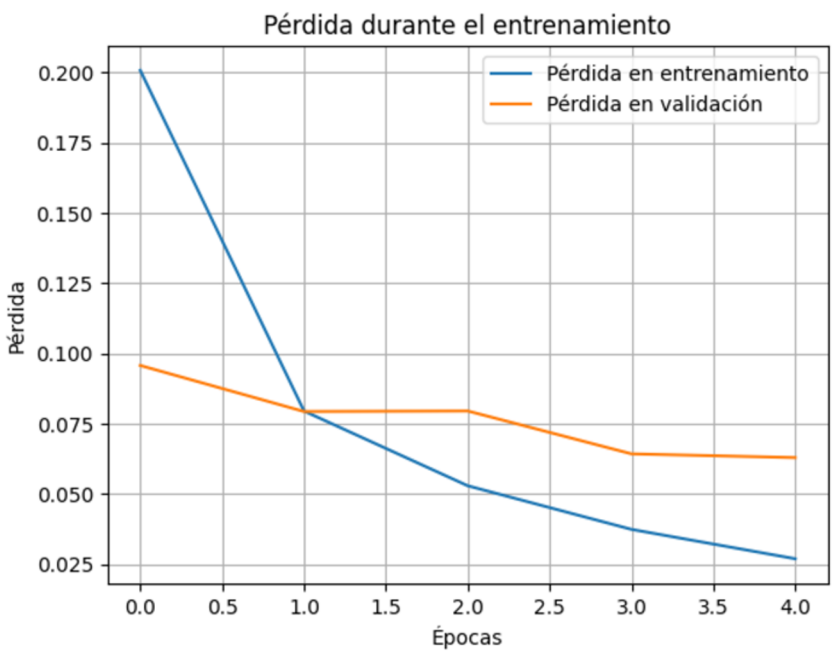
```
[ ]  # Evaluar el modelo en el conjunto de prueba
     test_loss, test_acc = model.evaluate(x_test, y_test)
     print('Precisión en el conjunto de prueba:', test_acc)

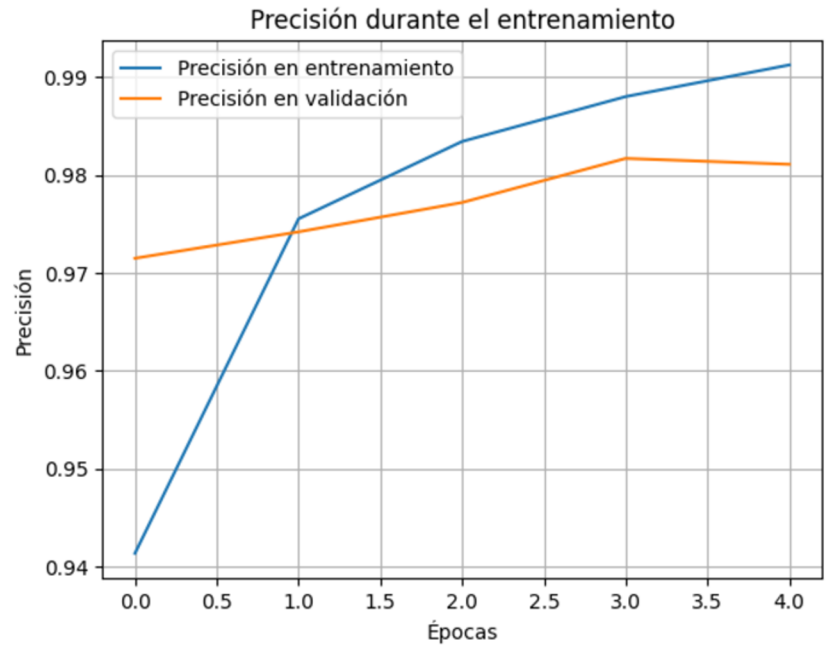
     313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9786 - loss: 0.0757
     Precisión en el conjunto de prueba: 0.9811000227928162
```

- La precisión del conjunto de prueba resulta ser del 97.8%, lo cual es *bastante* más bajo que la precisión del conjunto de entrenamiento (98.9%). Esta brecha entre la precisión del entrenamiento y la del test es un ejemplo de **overfitting**

Ajuste del modelo (Fitting)

Una forma útil de controlar si el modelo está overfitteando es mirar las curvas de entrenamiento y validación en cada época.

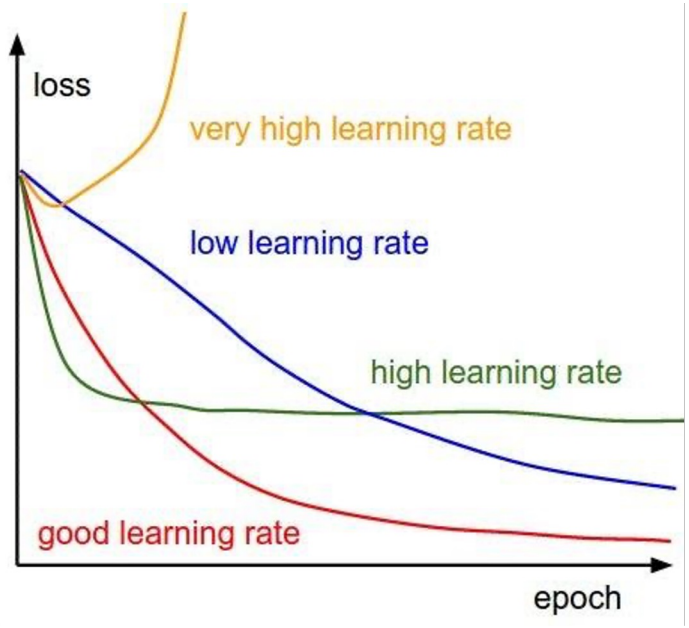




UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Ajuste del modelo (Fitting)

El **learning rate** afecta directamente a la forma de estas curvas. Learning rate bajo hace que el modelo demore en reducir la pérdida, pero muy altos hacen que el modelo no pueda aprender.



Inputs: Un mundo por explorar

Qué pasa si queremos utilizar como entrada otras cosas?

- Imágenes (más complejas).
- Audio.
- Video.
- Texto.

El problema siempre será **cómo** representar los datos en la capa de entrada y **cómo** definir la pérdida y/o métrica.

Adicionalmente veremos que hay otros tipos de capas intermedias que facilitan el *aprendizaje de las features* en otros tipos de datos.

- **Redes convolucionales**
- Mecanismos de atención (abre las puertas a LLMs)
- Redes recurrentes.